

问题：

你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有，是通过 Git 和 GitHub 进行代码管理，线上沟通分工合作的。

思考一下，Git 为什么要设计“暂存-提交”两个步骤？

暂存可以先 `add`；确认没问题，再 `commit`。这样提交更干净，好回溯查问题，不会把调试代码和正事混一起。改一半还能继续改，冲突也好处理。而且每次都提交太麻烦。

`git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

`git branch` 只看本地分支，`git branch -a` 本地和远程跟踪分支都会列出来。

文章大意和想法：

Commit message 和 Change log 编写指南

首先讲了 Commit message 的核心作用，用于规范提交信息，以及好处，提供更多的历史信息，方便快速浏览，可以过滤某些 commit（比如文档改动），便于快速查找信息和可以直接从 commit 生成 Change log。然后讲了格式，每次提交分三部分——Header（必需）、Body、Footer，之后介绍了一些工具，Commitizen，validate-commit-msg，最后讲了如何生成 Change log。

语义化版本

SemVer 的核心是用 主版本号.次版本号.修订号（X.Y.Z）这套格式，让版本号本身传递变更信息。

主版本号：做了不兼容的 API 修改时递增

次版本号：向下兼容的新功能时递增

修订号：向下兼容的问题修正时递增

它要解决的是“依赖地狱”：依赖太紧，升级一个包要连带改一堆；依赖太松，版本又会乱。

SemVer 通过一套明确的递增规则，让包管理工具能自动判断某个新版本是否安全可用。

前提是你得先有定义好的公共 API。只有 API 清晰了，版本号的改动才有意义——修改 API 就改版本号，看版本号就知道能不能升。

规范还回答了不少实操问题：`0.y.z` 是初始开发阶段，可以随便改 API；一旦被正式环境使用，就该发 `1.0.0`；如果不小心把不兼容改动当次版本号发了，不能改已发布的版本，只能发新版本修正。

SemVer 和 Git 其实在解决同一类问题：怎么让“变化”变得可沟通、可依赖。

SemVer 管的是“版本号说了什么”，Git 管的是“代码怎么变过来的”。两者都是现代软件协作的基础语言。

学 Git 不只是学命令。是学一套代码管理的思维方式：用 `commit` 记录意图，用分支隔离实验，用暂存区分拣改动，用 `tag` 标记发布版本。当每个人都用同一套方式表达自己的工作时，团队协作才会变得更好。

SemVer 让使用者一眼判断该不该升级依赖，Git 让协作者一眼看懂这次提交干了什么。两个工具合在一起，才构成一个可追溯、可回退、可协作的工程体系。所以学 Git，本质上是学怎么在一个多人开发的环境里，把自己的分工搞清楚。

实验步骤：

1. 通过模板仓库建立个人仓库

2. 通过 SSH 拷贝仓库：`cd ~`

`git clone git@github.com:1A2A3A4A5A-ac/ICS-Lab0.git`

`cd ICS-Lab0`

3. 打开 vscode code .，信任文件夹，修改 main.c，`git add main.c` 保存，`git commit -m "完成 main.c 中的 TODO 修改"`，`git push -u origin main`。

4. 新建 feature 分支并修改 main.c git checkout -b feature git add main.c git commit -m "feature 分支修改 main.c" git push origin feature 切换到 main, 修改 main.c git checkout main, git add main.c, git commit -m "main 分支修改 main.c", git push origin main
5. 合并 feature git merge feature, 出现

```
writing objects: 100% (3/3), 521 bytes | 521.00 KiB/s, done.  
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0  
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.  
To github.com:1A2A3A4A5A-ac/ICS-Lab0.git  
62011a1..3a0dafc main -> main  
hwz123456@hwz:~/ICS-Lab0$ git merge feature  
Auto-merging main.c  
CONFLICT (content): Merge conflict in main.c  
Automatic merge failed; fix conflicts and then commit the result.  
hwz123456@hwz:~/ICS-Lab0$
```

```
#include <stdio.h>  
  
int main()  
{  
    // @TODO: print a sentence you want.  
    printf("完成修改")  
采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更  
<<<<<< HEAD (当前更改)  
    printf("修改main分支")  
=====  
    printf('修改feature分支')  
>>>>>> feature (传入的更改)  
    printf("Hello, world!\n");  
}
```

此时手动改成我想要的就行了

git add main.c

git commit -m "merge feature 到 main 并解决冲突"

最后再存到 GitHub 上: git push origin main